

# **IN4MATX 285:**

# **Interactive Technology Studio**

**Programming: DOM  
Manipulation**

# Today's goals

By the end of today, you should be able to...

- Describe what programming adds to implementation of interfaces
- Explain the role of the Document Object Model (DOM)
- Write code which reads the DOM using JavaScript
- Write code which edits the DOM using JavaScript

# Language Roles

**HTML**



Markup  
language

**CSS**



Styling  
language

**JS**



Programming  
language

**Thus far, JavaScript (and coding in general) doesn't seem particularly useful for making interfaces.**

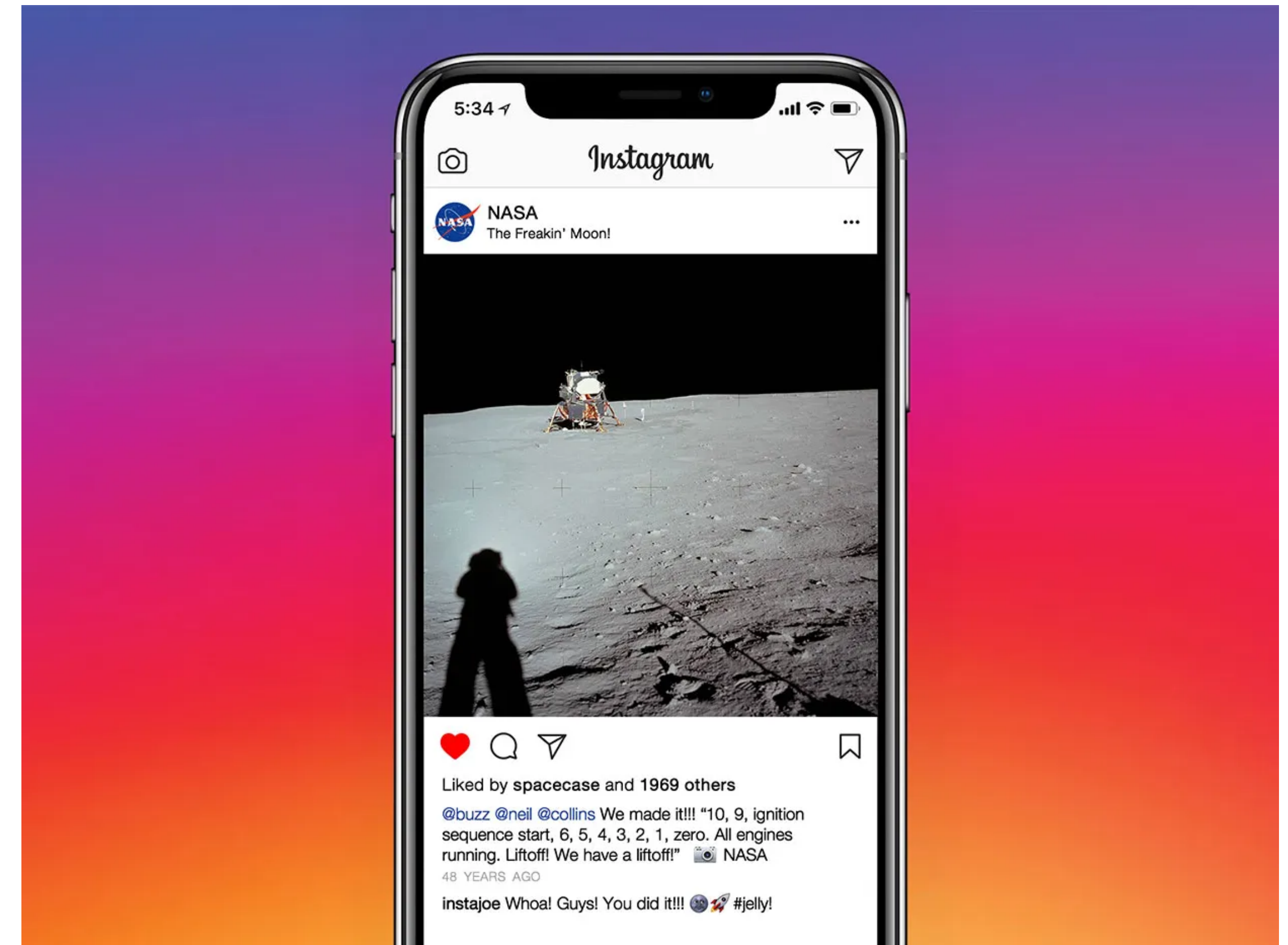
**Why** do we need programming in  
interfaces?

# Programming in Interfaces: Why?

- Dynamically change content
  - Reduce repetition across pages and experiences
- Parse and update in real time
  - In response to web data
- Handle user interaction
  - Button presses, text searches, etc.

# Programming in Interfaces: Why?

- Example: an Instagram feed
  - Lots of repetition: most of the layout is the same across users, but the images/videos vary (e.g., personalization)
  - Dynamic updating: comments on an Instagram Live need to propagate to other users
  - User interaction: what happens when you click the “heart” button?



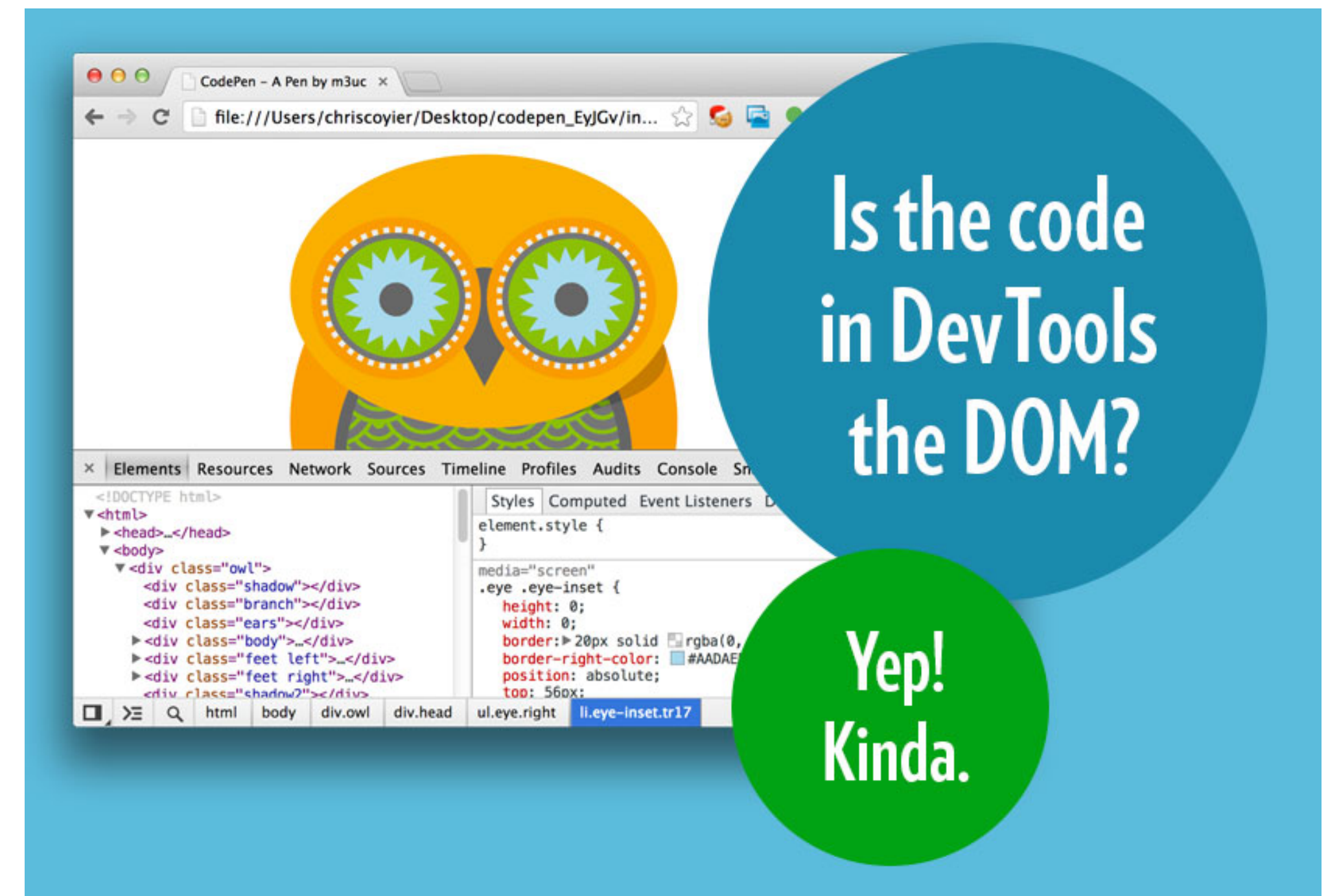
**Ok, so how do we code for dynamic interaction?**

# Coding for Dynamic Interaction in JS

- We'll use JavaScript to edit the HTML
  - In response to user interaction
  - In response to new web data as it comes in
  - To personalize content given the same “skeleton”

# HTML Document Object Model (DOM)

- A standard for how to get, change, add, or delete HTML elements
- The HTML you write is parsed by the browser and turned into the DOM
- JavaScript can then edit the DOM



<https://css-tricks.com/dom/>



# Demo: editing the DOM in your browser

**More often, you'll read and edit the  
DOM with JavaScript**

# Accessing the DOM in JavaScript

- document object model
- For example, you can write into the DOM with `document.write`
  - Appends to the end of the webpage

```
document.write("<h1>Hello, World!</h1>");
```

```
let myCourse = "IN4MATX 133";  
document.write("<h1>Hello, " + myCourse + " !");
```

# Example webpage

## HTML

```
<h1 id="title">Hi, 285 Students!</h1>
<p class="blue">This is a paragraph, written in blue.</p>
<p id="notblue">This paragraph is not blue.
    <strong class="blue">But, this bolded text is blue!</strong>
</p>
```

## CSS

```
.blue {
    color: blue;
}

.arial {
    font-family: Arial;
}
```

**Hi, 285 Students!**

This is a paragraph, written in blue.

This paragraph is not blue. **But, this bolded text is blue!**

# GetElementById

- Returns the one element with a matching id

```
let h1Title = document.getElementById( 'title' );  
console.log(h1Title);  
let notbluePara = document.getElementById( 'notblue' );  
console.log(notbluePara);
```

```
<h1 id="title">...</h1>  
<p class="blue">...</p>  
<p id="notblue">...  
<strong class="blue">...</strong>  
</p>
```

```
<h1 id="title">Hi, 285 Students!</h1>
```

---

```
▶ <p id="notblue">...</p>
```

# GetElementByClassName

- Returns **all** elements with a CSS class
- HTMLCollection is an *array*. We'll talk about arrays next time!

```
let blueTags = document.getElementsByClassName( 'blue' );  
console.log(blueTags);
```

```
▼ HTMLCollection(2) ⓘ  
  ▶ 0: p.blue  
  ▶ 1: strong.blue  
    length: 2  
  ▶ [[Prototype]]: HTMLCollection
```

```
<h1 id="title">...</h1>  
<p class="blue">...</p>  
<p id="notblue">...  
<strong class="blue">...</strong>  
</p>
```

# GetElementsByTagName

- Returns **all** elements with a particular tag
- HTMLCollection is an *array*. We'll talk about arrays next time!

```
let pTags = document.getElementsByTagName( 'p' );  
console.log( pTags );
```

```
<h1 id="title">...</h1>
```

```
<p class="blue">...</p>
```

```
<p id="notblue">...
```

```
<strong class="blue">...</strong>
```

```
</p>
```

```
▼ HTMLCollection(2) [p.blue, p#notblue]
```

```
‣ 0: p.blue
```

```
‣ 1: p#notblue
```

```
‣ notblue: p#notblue
```

```
length: 2
```

```
‣ [[Prototype]]: HTMLCollection
```

# Selecting elements

- We can reference individual HTML elements by calling selector functions

```
/*easiest to select by reusing CSS selectors! */
```

```
let cssSelector = 'header p, .title > p';
```

```
//selects FIRST element that matches css selector
```

```
let elem = document.querySelector(cssSelector);
```

```
//matches ALL elements that match css selector
```

```
let elems = document.querySelectorAll(cssSelector);
```

# Editing elements

```
let titleTag = document.getElementById('title');  
titleTag.textContent = 'This is a new title!';
```

```
let notBluePara = document.getElementById('notblue');  
notBluePara.innerHTML = 'And <em>this</em> is new HTML.';  
notBluePara.style.color = 'green';
```

**Hi, 285 Students!**

This is a paragraph, written in blue.

This paragraph is not blue. **But, this bolded text is blue!**

**This is a new title!**

This is a paragraph, written in blue.

And *this* is new HTML.

# Editing elements

- Can add/remove classes

```
let paraId = document.getElementById( 'para' );  
paraId.classList.add( 'arial' );
```

**Hi, 285 Students!**

This is a paragraph, written in blue.

**Hi, 285 Students!**

This is a paragraph, written in blue.

# Editing elements

## Hi, 285 Students!

This is a paragraph, written in blue.

This paragraph is not blue. **But, this bolded text is blue!**



## This is a new title!

This is a paragraph, written in blue.

And *this* is new HTML.

# Creating & Adding elements

```
//create a new <p> (not yet in the tree)
var newP = document.createElement('p');
newP.textContent = "I'm new!";

//create Node of textContent only
var newText = document.createTextNode("I'm blank");

var div = document.querySelector('div#container');
div.appendChild(newP); //add element INSIDE (at end)
div.appendChild(newText);

//add node BEFORE element (new, old)
div.insertBefore(document.createTextNode("First!"), newP);

//replace node (new, old)
div.replaceChild(document.createTextNode('boo'), newText);

//remove node
div.removeChild(div.querySelector('p'));
```

# Creating & Adding elements



## My shopping list

**Items to buy:**

# Adding listeners

- In HTML, you can specify a function to get called in response to a click action  
`<button onclick="myFunction()">Click me</button>`
- Will look for a function with the name `myFunction` in your JavaScript

# Adding listeners

- Listeners can also be added in code
- Can indicate a method which should be called when that element causes an event

```
//respond to "click" events  
elem.addEventListener('click', callback);
```

# Types of events

'click' //mouse or button clicked

'dblclick' //double-clicked

'hover' //mouse hover

'change' //an input changes (dropdown, radio)

'mouseenter' //mouse is moved over an element

'mouseleave' //mouse leaves the element

'mousedown' //mouse button is pressed

'keydown' //key is pressed

//... and more!

<https://developer.mozilla.org/en-US/docs/Web/Events>

# Handling events



## Toggle colors

Black ▾

**Black**

# Reflecting on DOM Manipulation

# Reflecting on DOM Manipulation

- There's **so** much that you can do via DOM Manipulation
  - Give elements new ids or other attributes
  - Edit or add to CSS Stylesheets
- Basically every interactive webpage leverages DOM Manipulation
  - Check your favorite webpage to see how it changes as you interact with it

# Today's goals

By the end of today, you should be able to...

- Describe what programming adds to implementation of interfaces
- Explain the role of the Document Object Model (DOM)
- Write code which reads the DOM using JavaScript
- Write code which edits the DOM using JavaScript

# **IN4MATX 285:**

# **Interactive Technology Studio**

**Programming: DOM  
Manipulation**

# **Additional Slides**

# Document ready: JavaScript

- The DOM won't be visible until the page has finished loading
  - Otherwise elements you're trying to access might not exist
  - `onload` function will run after the page has loaded

```
<head>
```

```
  <script>
```

```
    function pageLoaded( ) {  
      alert( "Page Loaded!" );  
    }
```

```
  </script>
```

```
</head>
```

```
<body onload="pageLoaded( )" >
```

```
</body>
```

# Event types

```
'click'          //mouse or button clicked  
'dblclick'       //double-clicked  
'hover'          //mouse hover  
'focus'         //element gains focus (important!)  
'mouseenter'     //mouse is moved over an element  
'mouseleave'     //mouse leaves the element  
'mousedown'      //mouse button is pressed  
'keydown'        //key is pressed  
  
//... and more!
```

<https://developer.mozilla.org/en-US/docs/Web/Events>

# Listeners

- Listener will be passed an **event** parameter

- Sometimes useful, but can often be ignored

```
elem.addEventListener('click', function(event) {  
    console.log('You clicked me!');
```

↑  
Remember, parameters are optional

```
    //get what was clicked;  
    let clickedElem = event.target;  
});
```

↑  
The “target” (source) of the event